

In-Class Assignment: Workflow Definition and LLM Workflow Selector

Goal

Build a workflow definition system and an LLM workflow selector.

By the end of this assignment, the backend can:

1. Create workflow definitions through API.
2. Store workflow definitions in DynamoDB.
3. List published workflow definitions.
4. Load workflow definitions from DynamoDB.
5. Send the user message and workflow definitions to OpenAI.
6. Select the best workflow.
7. Return a confidence score.
8. Extract required fields.
9. Return missing fields and follow-up questions.

Step 1: Create DynamoDB Table

Open AWS Console.

Go to:

```
DynamoDB
→ Tables
→ Create table
```

Create a table with this configuration:

```
Table name: WorkflowDefinitions
Partition key: workflow_id
Partition key type: String
Sort key: None
Table settings: Default settings
Capacity mode: On-demand
```

Wait until the table status becomes:

Active

Step 2: Update requirements.txt

Open:

```
requirements.txt
```

Update it to:

```
fastapi
uvicorn
pydantic
boto3
openai
```

Install dependencies:

```
pip install -r requirements.txt
```

Step 3: Set Environment Variables

Run:

```
export AWS_REGION="us-west-2"
export WORKFLOW_DEFINITIONS_TABLE_NAME="WorkflowDefinitions"
export OPENAI_API_KEY="YOUR_OPENAI_API_KEY"
export OPENAI_MODEL="gpt-5"
```

Step 4: Create app/config.py

Create:

```
app/config.py
```

Add:

```
import os

AWS_REGION = os.getenv("AWS_REGION", "us-west-2")

WORKFLOW_DEFINITIONS_TABLE_NAME = os.getenv(
    "WORKFLOW_DEFINITIONS_TABLE_NAME",
    "WorkflowDefinitions",
)

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-5")
```

Step 5: Create Workflow Definition Models

Create:

```
app/models/workflow_definition_models.py
```

Add:

```
from enum import Enum
from typing import List, Optional

from pydantic import BaseModel, Field

class WorkflowDefinitionStatus(str, Enum):
    DRAFT = "DRAFT"
    PUBLISHED = "PUBLISHED"
    DISABLED = "DISABLED"

class WorkflowFieldType(str, Enum):
    STRING = "STRING"
    NUMBER = "NUMBER"
    BOOLEAN = "BOOLEAN"
    ENUM = "ENUM"
```

```

class WorkflowFieldDefinition(BaseModel):
    name: str
    field_type: WorkflowFieldType
    description: str
    examples: List[str] = []
    required: bool = True
    allowed_values: Optional[List[str]] = None
    validation_regex: Optional[str] = None
    missing_field_question: Optional[str] = None

class CreateWorkflowDefinitionRequest(BaseModel):
    workflow_id: str = Field(..., min_length=1)
    workflow_type: str = Field(..., min_length=1)
    name: str = Field(..., min_length=1)
    description: str = Field(..., min_length=1)
    status: WorkflowDefinitionStatus = WorkflowDefinitionStatus.PUBLISHED
    version: int = 1
    trigger_examples: List[str] = []
    required_fields: List[WorkflowFieldDefinition] = []
    optional_fields: List[WorkflowFieldDefinition] = []
    steps: List[str] = []
    min_confidence_score: float = 0.7

class WorkflowDefinitionResponse(BaseModel):
    workflow_id: str
    workflow_type: str
    name: str
    description: str
    status: WorkflowDefinitionStatus
    version: int
    trigger_examples: List[str]
    required_fields: List[WorkflowFieldDefinition]
    optional_fields: List[WorkflowFieldDefinition]
    steps: List[str]
    min_confidence_score: float
    created_at: str
    updated_at: str

```

Step 6: Create Workflow Selector Models

Create:

```
app/models/workflow_selector_models.py
```

Add:

```
from typing import Dict, List, Optional

from pydantic import BaseModel, Field

class WorkflowSelectionRequest(BaseModel):
    message: str = Field(..., min_length=3)
    user_id: Optional[str] = None
    source: Optional[str] = "unknown"

class WorkflowSelectionResponse(BaseModel):
    selected_workflow_id: Optional[str]
    selected_workflow_type: Optional[str]
    confidence_score: float
    reason: str
    extracted_fields: Dict[str, str]
    missing_fields: List[str]
    missing_field_questions: List[str]
    is_ready_for_workflow: bool
```

Step 7: Create Workflow Definition Repository

Create:

```
app/repositories/workflow_definition_repository.py
```

Add:

```
from decimal import Decimal
from typing import List, Optional

import boto3
from boto3.dynamodb.conditions import Attr

from app.config import AWS_REGION, WORKFLOW_DEFINITIONS_TABLE_NAME
from app.models.workflow_definition_models import (
```

```

WorkflowDefinitionResponse,
WorkflowDefinitionStatus,
)

class WorkflowDefinitionRepository:
    def __init__(self):
        dynamodb = boto3.resource("dynamodb", region_name=AWS_REGION)
        self.table = dynamodb.Table(WORKFLOW_DEFINITIONS_TABLE_NAME)

    def save(
        self,
        workflow_definition: WorkflowDefinitionResponse,
    ) -> WorkflowDefinitionResponse:
        item = workflow_definition.model_dump(mode="json")

        item["min_confidence_score"] = Decimal(
            str(item["min_confidence_score"])
        )

        self.table.put_item(Item=item)

        return workflow_definition

    def get_by_id(
        self,
        workflow_id: str,
    ) -> Optional[WorkflowDefinitionResponse]:
        response = self.table.get_item(
            Key={
                "workflow_id": workflow_id
            }
        )

        item = response.get("Item")

        if item is None:
            return None

        return WorkflowDefinitionResponse(**item)

    def list_all(self) -> List[WorkflowDefinitionResponse]:
        response = self.table.scan()
        items = response.get("Items", [])

        return [
            WorkflowDefinitionResponse(**item)
            for item in items

```

```

    ]

    def list_published(self) -> List[WorkflowDefinitionResponse]:
        response = self.table.scan(
            FilterExpression=Attr("status").eq(
                WorkflowDefinitionStatus.PUBLISHED.value
            )
        )

        items = response.get("Items", [])

        return [
            WorkflowDefinitionResponse(**item)
            for item in items
        ]

```

Step 8: Create Workflow Definition Service

Create:

```
app/services/workflow_definition_service.py
```

Add:

```

from datetime import datetime, timezone
from typing import List, Optional

from app.models.workflow_definition_models import (
    CreateWorkflowDefinitionRequest,
    WorkflowDefinitionResponse,
)
from app.repositories.workflow_definition_repository import (
    WorkflowDefinitionRepository,
)

class WorkflowDefinitionService:
    def __init__(self):
        self.workflow_definition_repository = WorkflowDefinitionRepository()

    def create_workflow_definition(
        self,
        request: CreateWorkflowDefinitionRequest,

```

```

) -> WorkflowDefinitionResponse:
    now = self._current_time_iso()

    workflow_definition = WorkflowDefinitionResponse(
        workflow_id=request.workflow_id,
        workflow_type=request.workflow_type,
        name=request.name,
        description=request.description,
        status=request.status,
        version=request.version,
        trigger_examples=request.trigger_examples,
        required_fields=request.required_fields,
        optional_fields=request.optional_fields,
        steps=request.steps,
        min_confidence_score=request.min_confidence_score,
        created_at=now,
        updated_at=now,
    )

    return self.workflow_definition_repository.save(workflow_definition)

def get_workflow_definition(
    self,
    workflow_id: str,
) -> Optional[WorkflowDefinitionResponse]:
    return self.workflow_definition_repository.get_by_id(workflow_id)

def list_workflow_definitions(self) -> List[WorkflowDefinitionResponse]:
    return self.workflow_definition_repository.list_all()

def list_published_workflow_definitions(self) ->
List[WorkflowDefinitionResponse]:
    return self.workflow_definition_repository.list_published()

def _current_time_iso(self) -> str:
    return datetime.now(timezone.utc).isoformat()

workflow_definition_service = WorkflowDefinitionService()

```

Step 9: Create Workflow Definition Router

Create:

```
app/routers/workflow_definition_router.py
```

Add:

```
from typing import List

from fastapi import APIRouter, HTTPException, status

from app.models.workflow_definition_models import (
    CreateWorkflowDefinitionRequest,
    WorkflowDefinitionResponse,
)
from app.services.workflow_definition_service import workflow_definition_service

router = APIRouter(
    prefix="/workflow-definitions",
    tags=["Workflow Definitions"],
)

@router.post(
    "",
    response_model=WorkflowDefinitionResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_workflow_definition(request: CreateWorkflowDefinitionRequest):
    return workflow_definition_service.create_workflow_definition(request)

@router.get(
    "",
    response_model=List[WorkflowDefinitionResponse],
)
def list_workflow_definitions():
    return workflow_definition_service.list_workflow_definitions()

@router.get(
    "/published",
    response_model=List[WorkflowDefinitionResponse],
)
def list_published_workflow_definitions():
    return workflow_definition_service.list_published_workflow_definitions()
```

```

@router.get(
   ("/{workflow_id}",
    response_model=WorkflowDefinitionResponse,
)
def get_workflow_definition(workflow_id: str):
    workflow_definition = workflow_definition_service.get_workflow_definition(
        workflow_id
    )

    if workflow_definition is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Workflow definition not found",
        )

    return workflow_definition

```

Step 10: Create LLM Workflow Selector Service

Create:

```
app/services/llm_workflow_selector_service.py
```

Add:

```

import json
from typing import Dict, List, Optional

from openai import OpenAI

from app.config import OPENAI_MODEL
from app.models.workflow_definition_models import WorkflowDefinitionResponse
from app.models.workflow_selector_models import (
    WorkflowSelectionRequest,
    WorkflowSelectionResponse,
)
from app.services.workflow_definition_service import workflow_definition_service

class LLMWorkflowSelectorService:
    def __init__(self):
        self.client = OpenAI()

```

```

def select_workflow(
    self,
    request: WorkflowSelectionRequest,
) -> WorkflowSelectionResponse:
    workflow_definitions = (
        workflow_definition_service.list_published_workflow_definitions()
    )

    if not workflow_definitions:
        return WorkflowSelectionResponse(
            selected_workflow_id=None,
            selected_workflow_type=None,
            confidence_score=0.0,
            reason="No published workflow definitions are available.",
            extracted_fields={},
            missing_fields=[],
            missing_field_questions=[],
            is_ready_for_workflow=False,
        )

    raw_selection = self._call_openai(
        user_message=request.message,
        workflow_definitions=workflow_definitions,
    )

    return self._build_response(
        raw_selection=raw_selection,
        workflow_definitions=workflow_definitions,
    )

def _call_openai(
    self,
    user_message: str,
    workflow_definitions: List[WorkflowDefinitionResponse],
) -> Dict:
    system_prompt = """

```

You are a workflow selection engine for a client issue triage platform.

Your job:

1. Read the user's message.
2. Review all available workflow definitions.
3. Select the best matching workflow.
4. Extract required and optional fields from the user message.
5. Identify missing required fields.
6. Return a confidence score between 0 and 1.

Rules:

- Only select a workflow from the provided workflow definitions.

- If none of the workflows match, selected_workflow_id must be null.
- Do not invent extracted field values.
- If a required field is missing, include it in missing_fields.
- Use the workflow definition's missing_field_question when possible.
- confidence_score is your estimated confidence in the selected workflow.

```

workflow_definitions_text = self._format_workflow_definitions(
    workflow_definitions
)

```

```

user_prompt = f"""

```

User message:

```
{user_message}
```

Available workflow definitions:

```
{workflow_definitions_text}
```

Return the workflow selection result as JSON.

```

"""

```

```

response = self.client.responses.create(
    model=OPENAI_MODEL,
    input=[
        {
            "role": "system",
            "content": system_prompt,
        },
        {
            "role": "user",
            "content": user_prompt,
        },
    ],
    text={
        "format": {
            "type": "json_schema",
            "name": "workflow_selection_result",
            "strict": True,
            "schema": {
                "type": "object",
                "additionalProperties": False,
                "properties": {
                    "selected_workflow_id": {
                        "type": ["string", "null"]
                    },
                    "selected_workflow_type": {
                        "type": ["string", "null"]
                    },
                },
            },
        },
    },
)

```

```

        "confidence_score": {
            "type": "number"
        },
        "reason": {
            "type": "string"
        },
        "extracted_fields": {
            "type": "array",
            "items": {
                "type": "object",
                "additionalProperties": False,
                "properties": {
                    "field_name": {
                        "type": "string"
                    },
                    "field_value": {
                        "type": "string"
                    }
                },
                "required": [
                    "field_name",
                    "field_value",
                ],
            },
        },
        "missing_fields": {
            "type": "array",
            "items": {
                "type": "string"
            },
        },
        "missing_field_questions": {
            "type": "array",
            "items": {
                "type": "string"
            },
        },
    },
    "required": [
        "selected_workflow_id",
        "selected_workflow_type",
        "confidence_score",
        "reason",
        "extracted_fields",
        "missing_fields",
        "missing_field_questions",
    ],
},

```

```

        }
    },
)

return json.loads(response.output_text)

def _format_workflow_definitions(
    self,
    workflow_definitions: List[WorkflowDefinitionResponse],
) -> str:
    simplified_definitions = []

    for workflow in workflow_definitions:
        simplified_definitions.append(
            {
                "workflow_id": workflow.workflow_id,
                "workflow_type": workflow.workflow_type,
                "name": workflow.name,
                "description": workflow.description,
                "trigger_examples": workflow.trigger_examples,
                "required_fields": [
                    field.model_dump(mode="json")
                    for field in workflow.required_fields
                ],
                "optional_fields": [
                    field.model_dump(mode="json")
                    for field in workflow.optional_fields
                ],
                "min_confidence_score": workflow.min_confidence_score,
            }
        )

    return json.dumps(simplified_definitions, indent=2)

def _build_response(
    self,
    raw_selection: Dict,
    workflow_definitions: List[WorkflowDefinitionResponse],
) -> WorkflowSelectionResponse:
    selected_workflow_id = raw_selection.get("selected_workflow_id")
    selected_workflow = self._find_workflow_definition(
        selected_workflow_id=selected_workflow_id,
        workflow_definitions=workflow_definitions,
    )

    confidence_score = float(raw_selection.get("confidence_score", 0.0))
    missing_fields = raw_selection.get("missing_fields", [])

```

```

        extracted_fields = self._normalize_extracted_fields(
            raw_selection.get("extracted_fields", [])
        )

        is_ready_for_workflow = False

        if selected_workflow is not None:
            is_ready_for_workflow = (
                confidence_score >= selected_workflow.min_confidence_score
                and len(missing_fields) == 0
            )

        return WorkflowSelectionResponse(
            selected_workflow_id=raw_selection.get("selected_workflow_id"),
            selected_workflow_type=raw_selection.get("selected_workflow_type"),
            confidence_score=confidence_score,
            reason=raw_selection.get("reason", ""),
            extracted_fields=extracted_fields,
            missing_fields=missing_fields,
            missing_field_questions=raw_selection.get(
                "missing_field_questions",
                [],
            ),
            is_ready_for_workflow=is_ready_for_workflow,
        )

    def _normalize_extracted_fields(self, raw_fields: List[Dict]) -> Dict[str,
str]:
        extracted_fields = {}

        for field in raw_fields:
            field_name = field.get("field_name")
            field_value = field.get("field_value")

            if field_name and field_value:
                extracted_fields[field_name] = field_value

        return extracted_fields

    def _find_workflow_definition(
        self,
        selected_workflow_id: Optional[str],
        workflow_definitions: List[WorkflowDefinitionResponse],
    ) -> Optional[WorkflowDefinitionResponse]:
        if selected_workflow_id is None:
            return None

        for workflow in workflow_definitions:

```

```
        if workflow.workflow_id == selected_workflow_id:
            return workflow

    return None

llm_workflow_selector_service = LLMWorkflowSelectorService()
```

Step 11: Create Workflow Selector Router

Create:

```
app/routers/workflow_selector_router.py
```

Add:

```
from fastapi import APIRouter

from app.models.workflow_selector_models import (
    WorkflowSelectionRequest,
    WorkflowSelectionResponse,
)
from app.services.llm_workflow_selector_service import (
    llm_workflow_selector_service,
)

router = APIRouter(
    prefix="/workflow-selector",
    tags=["Workflow Selector"],
)

@router.post(
    "/select",
    response_model=WorkflowSelectionResponse,
)
def select_workflow(request: WorkflowSelectionRequest):
    return llm_workflow_selector_service.select_workflow(request)
```

Step 12: Update app/main.py

Update:

```
app/main.py
```

Use:

```
from fastapi import FastAPI

from app.routers.ticket_router import router as ticket_router
from app.routers.workflow_definition_router import (
    router as workflow_definition_router,
)
from app.routers.workflow_selector_router import (
    router as workflow_selector_router,
)

app = FastAPI(
    title="AI Ticket Workflow Platform",
    description="A backend platform for client issue triage and workflow automation.",
    version="0.3.0",
)

app.include_router(ticket_router)
app.include_router(workflow_definition_router)
app.include_router(workflow_selector_router)

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }
```

Step 13: Start Server

Run:

```
uvicorn app.main:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

Step 14: Create Workflow Definition 1

Call:

```
POST /workflow-definitions
```

Request body:

```
{
  "workflow_id": "wf_stock_data_issue_v1",
  "workflow_type": "STOCK_DATA_ISSUE",
  "name": "Stock Data Issue Workflow",
  "description": "Handles reports about stale, delayed, missing, or incorrect
stock market data, including price, volume, percent change, market cap, or chart
data.",
  "status": "PUBLISHED",
  "version": 1,
  "trigger_examples": [
    "Tesla price has not updated for 30 minutes",
    "AAPL chart shows the wrong price",
    "NVDA volume looks incorrect",
    "The price data is delayed"
  ],
  "required_fields": [
    {
      "name": "symbol",
      "field_type": "STRING",
      "description": "The stock ticker symbol affected by the issue.",
      "examples": [
        "TSLA",
        "AAPL",
        "NVDA"
      ],
      "required": true,
      "allowed_values": null,
    }
  ]
}
```

```

    "validation_regex": "^[A-Z]{1,5}$",
    "missing_field_question": "Which stock symbol has the issue?"
  },
  {
    "name": "issue_description",
    "field_type": "STRING",
    "description": "A short description of the stock data problem the user is
reporting.",
    "examples": [
      "price not updating",
      "volume is wrong",
      "chart is frozen"
    ],
    "required": true,
    "allowed_values": null,
    "validation_regex": null,
    "missing_field_question": "What issue is the user seeing with the stock
data?"
  }
],
"optional_fields": [
  {
    "name": "time_window",
    "field_type": "STRING",
    "description": "How long the issue has been happening.",
    "examples": [
      "30 minutes",
      "since market open"
    ],
    "required": false,
    "allowed_values": null,
    "validation_regex": null,
    "missing_field_question": null
  }
],
"steps": [
  "VALIDATE_STOCK_DATA",
  "CREATE_ENGINEERING_TICKET",
  "NOTIFY_SUPPORT"
],
"min_confidence_score": 0.7
}

```

Expected response:

201 Created

Step 15: Create Workflow Definition 2

Call:

`POST /workflow-definitions`

Request body:

```
{
  "workflow_id": "wf_watchlist_bug_v1",
  "workflow_type": "WATCHLIST_BUG",
  "name": "Watchlist Bug Workflow",
  "description": "Handles issues where the user's watchlist is missing,
incorrect, not syncing, or cannot add or remove stocks.",
  "status": "PUBLISHED",
  "version": 1,
  "trigger_examples": [
    "My watchlist disappeared",
    "I cannot add TSLA to my watchlist",
    "My watchlist order is wrong",
    "Stocks I removed are still showing in my watchlist"
  ],
  "required_fields": [
    {
      "name": "watchlist_issue",
      "field_type": "STRING",
      "description": "The watchlist problem reported by the user.",
      "examples": [
        "watchlist disappeared",
        "cannot add stock",
        "watchlist order is wrong"
      ],
      "required": true,
      "allowed_values": null,
      "validation_regex": null,
      "missing_field_question": "What problem are you seeing with your
watchlist?"
    }
  ],
  "optional_fields": [
```

```

{
  "name": "symbol",
  "field_type": "STRING",
  "description": "The stock ticker symbol involved in the watchlist issue,
if any.",
  "examples": [
    "TSLA",
    "AAPL"
  ],
  "required": false,
  "allowed_values": null,
  "validation_regex": null,
  "missing_field_question": null
}
],
"steps": [
  "CHECK_WATCHLIST_STATE",
  "CREATE_ENGINEERING_TICKET",
  "NOTIFY_SUPPORT"
],
"min_confidence_score": 0.7
}

```

Expected response:

201 Created

Step 16: Create Workflow Definition 3

Call:

POST /workflow-definitions

Request body:

```

{
  "workflow_id": "wf_general_product_question_v1",
  "workflow_type": "GENERAL_PRODUCT_QUESTION",
  "name": "General Product Question Workflow",
  "description":
    "Handles general product questions, how-to questions, and basic troubleshooting

```

```

questions that may be answered using product knowledge or support
documentation.",
  "status": "PUBLISHED",
  "version": 1,
  "trigger_examples": [
    "How do I create a watchlist?",
    "Where can I see after-hours prices?",
    "Why does a stock price change after market close?",
    "How do I view my portfolio performance?"
  ],
  "required_fields": [
    {
      "name": "question",
      "field_type": "STRING",
      "description": "The user's product or troubleshooting question.",
      "examples": [
        "How do I create a watchlist?",
        "Where can I see after-hours prices?"
      ],
      "required": true,
      "allowed_values": null,
      "validation_regex": null,
      "missing_field_question": "What product question does the user have?"
    }
  ],
  "optional_fields": [],
  "steps": [
    "SEARCH_KNOWLEDGE_BASE",
    "GENERATE_SUPPORT_REPLY"
  ],
  "min_confidence_score": 0.65
}

```

Expected response:

201 Created

Step 17: Verify Workflow Definitions

Call:

```
GET /workflow-definitions
```

Expected result:

```
Returns 3 workflow definitions.
```

Call:

```
GET /workflow-definitions/published
```

Expected result:

```
Returns 3 published workflow definitions.
```

Call:

```
GET /workflow-definitions/wf_stock_data_issue_v1
```

Expected result:

```
Returns the Stock Data Issue Workflow definition.
```

Step 18: Test LLM Workflow Selector

Test 1: Stock data issue

Call:

```
POST /workflow-selector/select
```

Request body:

```
{  
  "message": "TSLA price has not updated for 30 minutes.",  
  "user_id": "user_123",  
}
```

```
"source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_stock_data_issue_v1",
  "selected_workflow_type": "STOCK_DATA_ISSUE",
  "confidence_score": 0.8,
  "reason": "...",
  "extracted_fields": {
    "symbol": "TSLA",
    "issue_description": "...",
    "time_window": "30 minutes"
  },
  "missing_fields": [],
  "missing_field_questions": [],
  "is_ready_for_workflow": true
}
```

Test 2: Watchlist bug

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "My watchlist disappeared this morning.",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_watchlist_bug_v1",
  "selected_workflow_type": "WATCHLIST_BUG",
  "confidence_score": 0.8,
  "reason": "...",
}
```

```
"extracted_fields": {
  "watchlist_issue": "watchlist disappeared this morning"
},
"missing_fields": [],
"missing_field_questions": [],
"is_ready_for_workflow": true
}
```

Test 3: Product question

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "Where can I see after-hours stock prices?",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_general_product_question_v1",
  "selected_workflow_type": "GENERAL_PRODUCT_QUESTION",
  "confidence_score": 0.8,
  "reason": "...",
  "extracted_fields": {
    "question": "Where can I see after-hours stock prices?"
  },
  "missing_fields": [],
  "missing_field_questions": [],
  "is_ready_for_workflow": true
}
```

Test 4: Missing required field

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "The stock price is wrong.",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_stock_data_issue_v1",
  "selected_workflow_type": "STOCK_DATA_ISSUE",
  "confidence_score": 0.7,
  "reason": "...",
  "extracted_fields": {
    "issue_description": "stock price is wrong"
  },
  "missing_fields": [
    "symbol"
  ],
  "missing_field_questions": [
    "Which stock symbol has the issue?"
  ],
  "is_ready_for_workflow": false
}
```

Step 19: Check DynamoDB

Open AWS Console.

Go to:

```
DynamoDB
→ Tables
→ WorkflowDefinitions
→ Explore table items
```

Confirm these items exist:

```
wf_stock_data_issue_v1  
wf_watchlist_bug_v1  
wf_general_product_question_v1
```

Step 20: Commit Code

Run:

```
git status  
git add .  
git commit -m "Add workflow definitions and LLM selector"  
git push origin main
```

Acceptance Criteria

The assignment is complete when:

1. `WorkflowDefinitions` table exists in DynamoDB.
 2. `POST /workflow-definitions` creates a workflow definition.
 3. `GET /workflow-definitions` returns all workflow definitions.
 4. `GET /workflow-definitions/published` returns published workflow definitions.
 5. Three workflow definitions are created through API requests.
 6. Workflow definitions are stored in DynamoDB.
 7. `POST /workflow-selector/select` calls OpenAI.
 8. The selector returns `selected_workflow_id`.
 9. The selector returns `selected_workflow_type`.
 10. The selector returns `confidence_score`.
 11. The selector returns `extracted_fields`.
 12. The selector returns `missing_fields`.
 13. The selector returns `missing_field_questions`.
 14. `is_ready_for_workflow` is `true` when confidence is high enough and all required fields are extracted.
 15. `is_ready_for_workflow` is `false` when required fields are missing.
-

Questions

1. What is a workflow definition?
2. Why do we store workflow definitions in DynamoDB?
3. What is `workflow_type` used for?
4. What are `trigger_examples` used for?
5. What are `required_fields` used for?
6. What does `confidence_score` mean?
7. What does `is_ready_for_workflow` mean?
8. Why does the selector return missing fields?
9. How can we add a new workflow type?
10. Why is it useful that the selector code does not hardcode workflow types?